

Constructing the DOS Platform UI

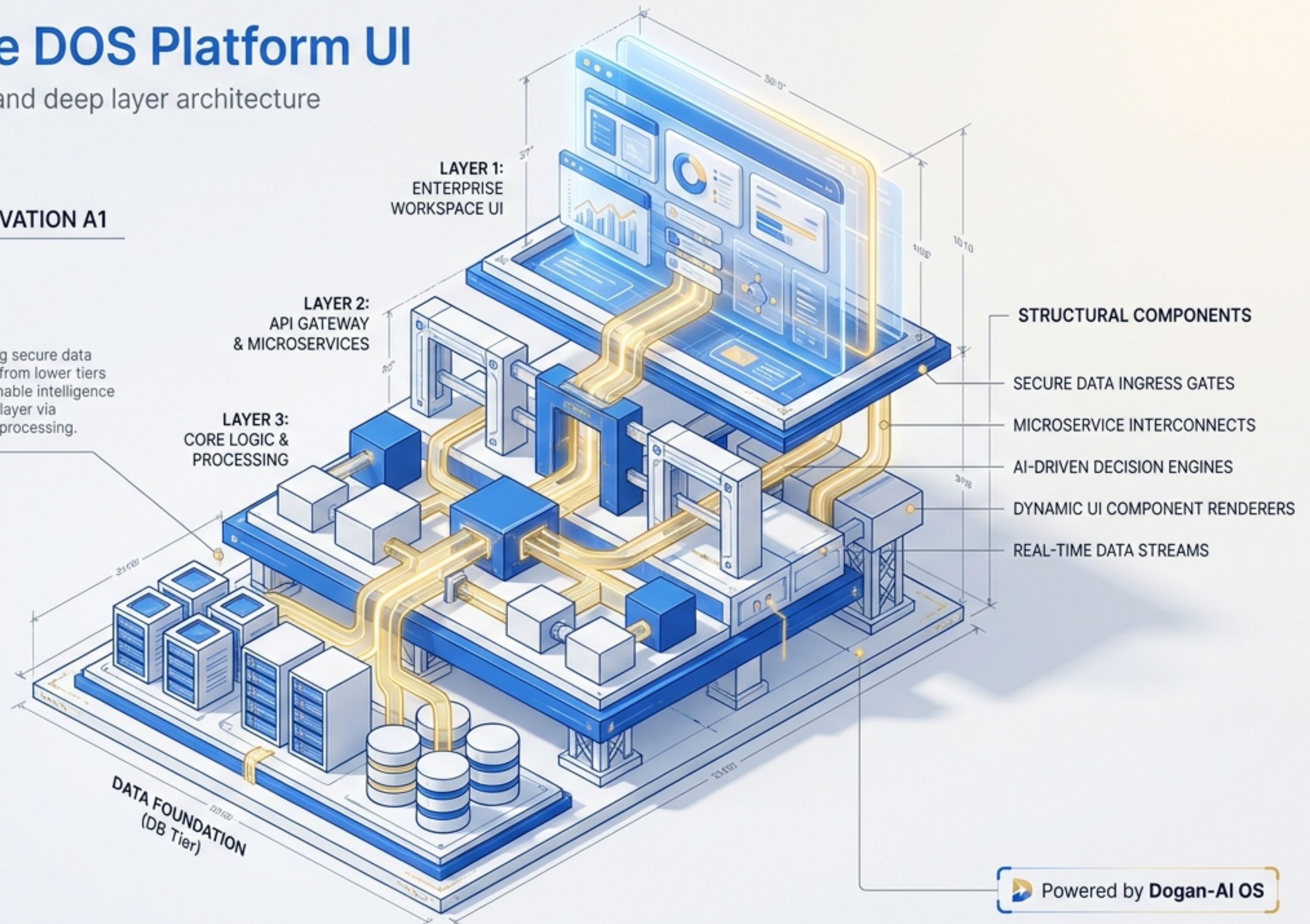
Tracing the dynamic data flow and deep layer architecture of an enterprise workspace

SYSTEM ARCHITECTURE PLAN - ELEVATION A1

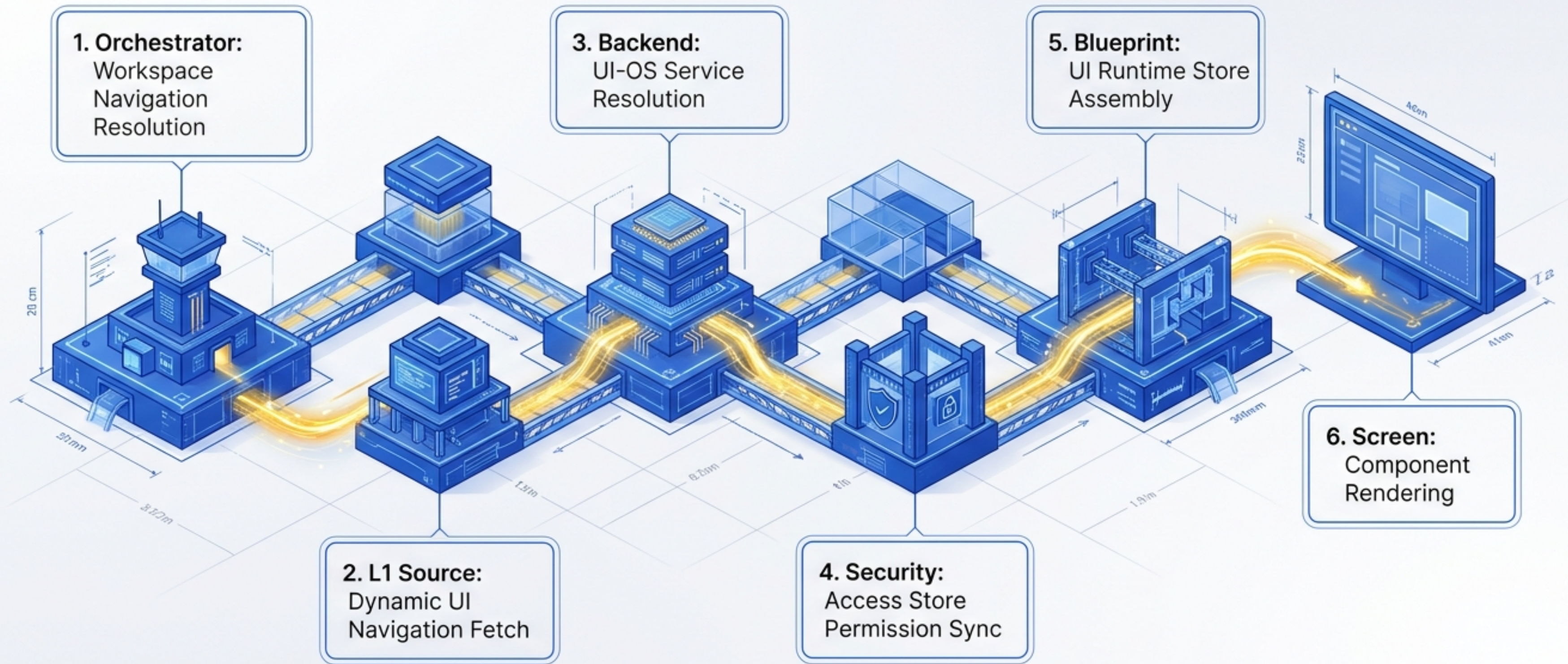
CODE-TO-CONCEPT BRIDGE

```
1 <data_pipeline>
2   <node id='ingress'>
3     <source>DB_Tierv</source>
4     <target>Core_Loglcc</target>
5     <status>ACTIVE</status>
6   </node>
7   <process id='transform'>
8     <input>raw_data</input>
9     <output>structured_insights</output>
10    <engine>AI_GS_Engine</engine>
11  </process>
12 </data_pipeline>
```

Translating secure data ingestion from lower tiers into actionable intelligence for the UI layer via AI-driven processing.



The Data Flow Journey



The 6-Layer Orchestrator

`WorkspaceNavigationAdapter.refresh()`

Node 1: L1 Dynamic UI
(Awaited sequentially)

Node 2:
Platform DNA

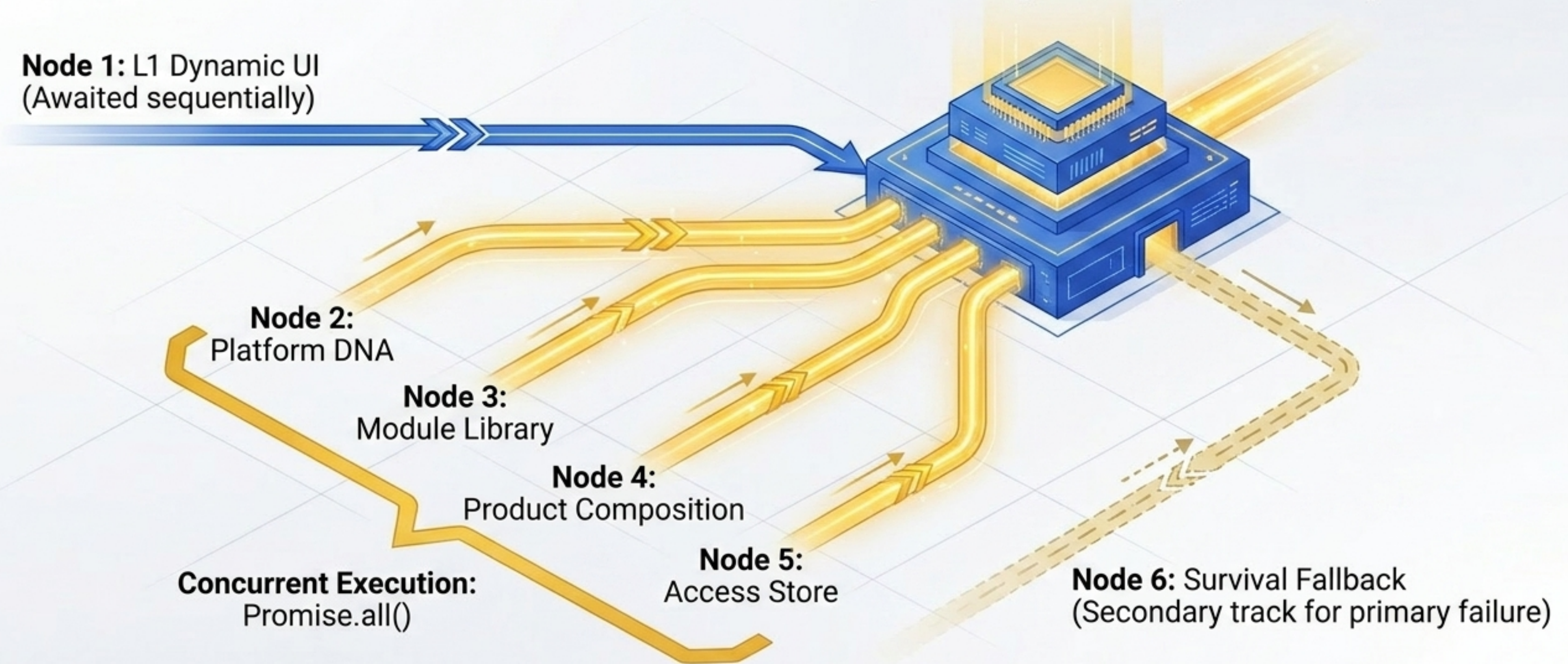
Node 3:
Module Library

Node 4:
Product Composition

Node 5:
Access Store

Concurrent Execution:
`Promise.all()`

Node 6: Survival Fallback
(Secondary track for primary failure)



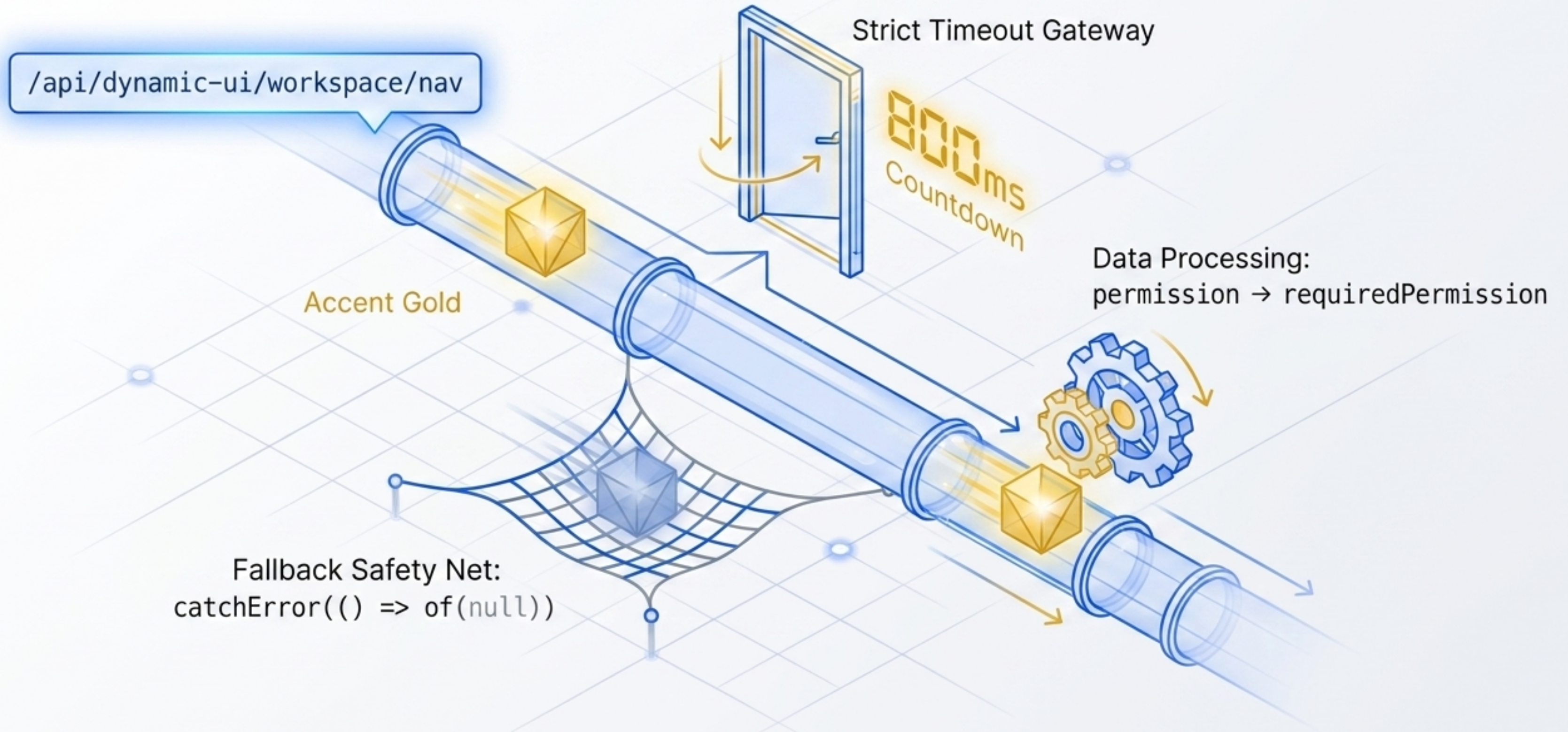
Navigation Layer Precedence Matrix

	Layer	Source Origin	Resolution Mode	Failure Tolerance
L1	Dynamic UI	Backend Database	Sequential	Catch Error to Null
L2	Platform DNA	Client Configuration	Concurrent	Catch Error to Null
L3	Module Library	Client Configuration	Concurrent	Catch Error to Null
L4	Product Comp.	Client Configuration	Concurrent	Catch Error to Null
L5	Access Store	Client State	Concurrent	Catch Error to Null
L6	Fallback	Fallback Config	Sequential	Hard Fail

Key Insight: Layers 2-5 resolve concurrently to eliminate network bottlenecks, while L1 initiates the primary structural fetch.



The Dynamic UI L1 Fetch



UI-OS Database Contract Resolution

Ring 1: Context Extraction

Extracts tenantId from principal identity or x-dos-tenant-id header

Secure Output Flow

Constructed navigation items securely flow back to the frontend

Ring 2: Query Execution & Filtering

Rejects rows where tenant_id does not match or readiness is not active



AccessStore Secure Vault

Access Store Checkpoint

Stream 1: fetchPermissions()
→ Sets `_permissions` and `_modules`

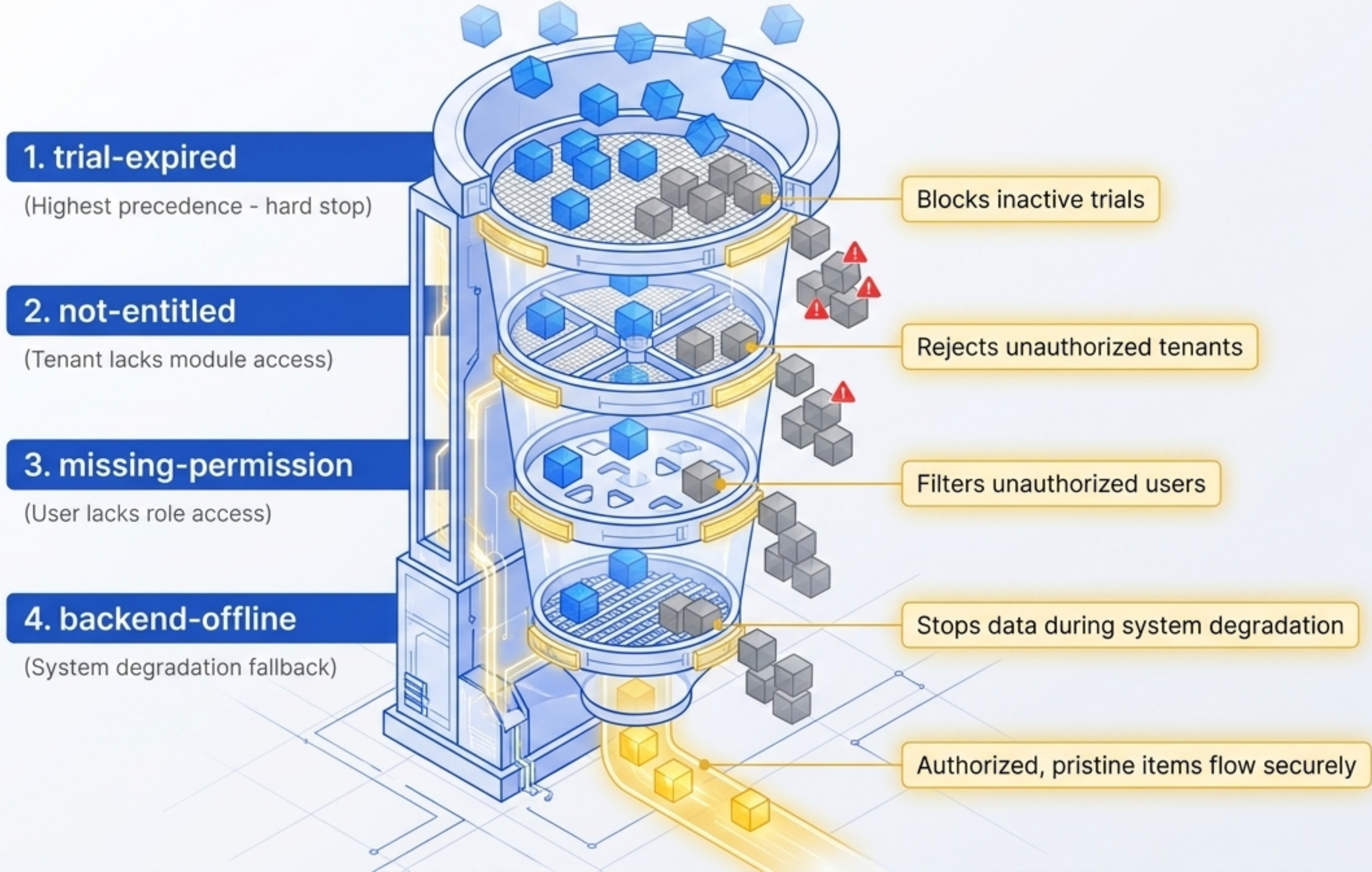
Stream 2: fetchMe()
→ Establishes user context

Stream 3: fetchTrialSummary()
→ Tracks `_trialExpired` modules

Signal-based reactive
updates radiating outward



The Disabled Reason Funnel



Blueprint Assembly Line

Raw Inputs:

Tenant Data

Access Data
(buildPermissionSet())

Active Modules
(entitlements())

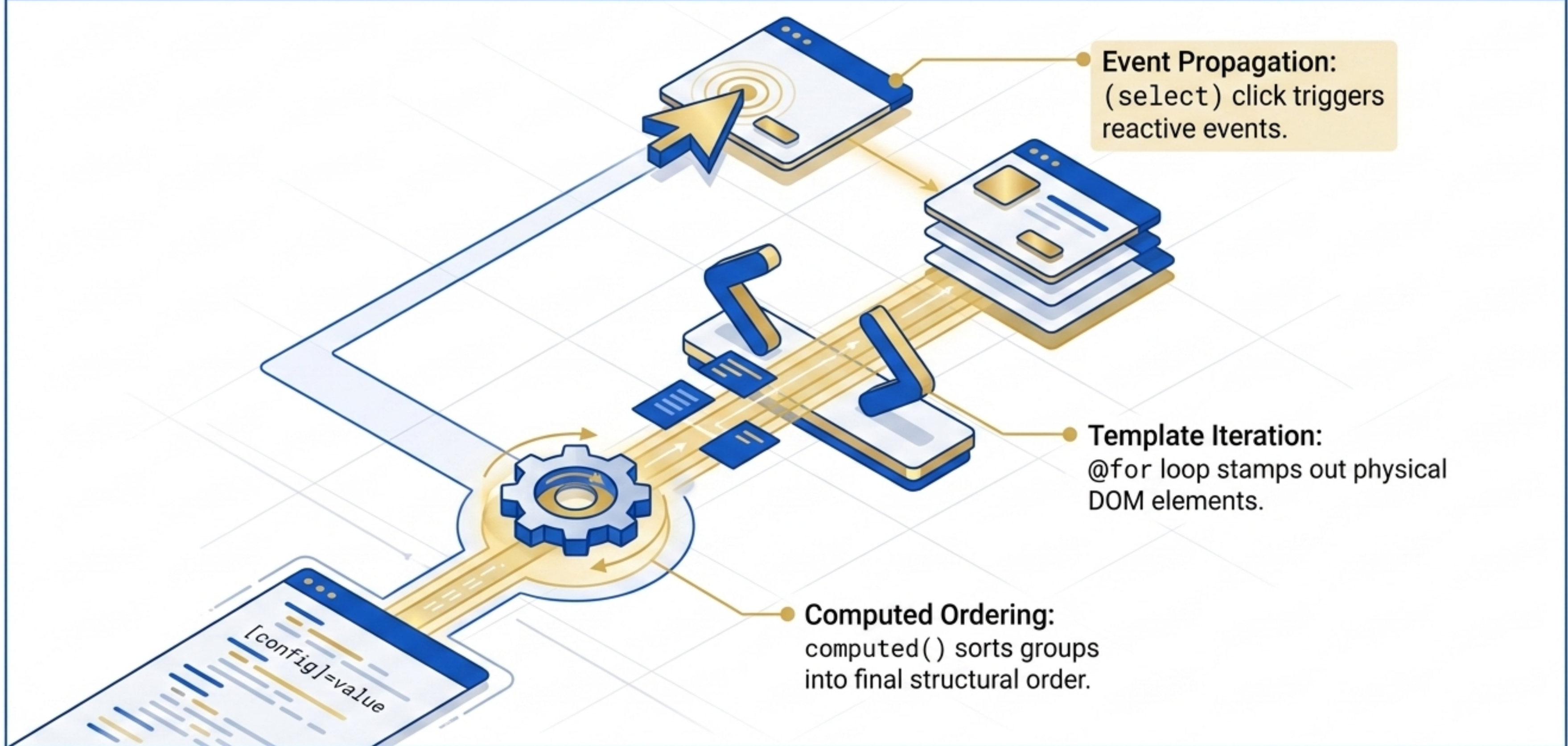
The Press:
`ui-runtime-store.service.ts`

Output:
EffectiveUiState Blueprint

Signal-based reactive blueprint published.



Component Rendering Execution



Governance by Design

The **DOS Platform UI** does not just hide buttons. It structurally **forbids unauthorized actions**. By requiring **6-layer orchestration**, **tenant database isolation**, and **parallel permission checks** before rendering a single pixel, the UI guarantees **strict Segregation of Duties (SoD)** and **identity compliance**.



Top Layer:
Presentation UI
(Blindly renders the blueprint)

Middle Layer:
DAuth Access Store
(The strict gatekeeper)

Bottom Layer:
UI-OS Backend
(Isolated tenant data)

One OS Architecture. Infinite Enterprise Interfaces.

The dynamic data flow architecture of Dogan-AI OS allows heavily regulated environments to generate deeply governed, tenant-isolated workspaces without hardcoding security logic into the presentation layer.

